ACSC430 Python cheat sheet

STRING SLICING & METHODS

```
s = '1357 Country Ln.'
s[4:]   → ' Country Ln.'
s[:4]   → '1357'
s[2:5]  → '57 '
```

s.rstrip('.')	remove from end
s.strip()	remove both ends
s.find('sub')	index or -1
s.startswith('x')	True/False
s.endswith('x')	True/False

⚠ find() checks presence; startswith/endswith do not search the middle.

LOOPS

```
fruits = ["apple","banana","cherry"]

for x in fruits:        # iterates all
  print(x)

for x in fruits:
  if x == "banana":
    break          # outputs: apple
  print(x)

for x in fruits:
  if x == "banana":
    continue       # skips banana
  print(x)         # apple, cherry
```

condition-controlled	while condition:
count-controlled	for i in range(n):

Priming read = first input read *before* the validation loop starts.

CONDITIONALS

```
if x == "banana":
  break
elif x == "apple":
  print("apple")
elif x != "cherry":
  pass
else:
  print("other")
```

not equal	!= (not <>)
must end with colon	if x != 10:
compound expression	and / or / not

Combining two Boolean expressions with and / or creates a **compound** expression.

LISTS & DICTS

```
ages = {'Aaron':6,'Kelly':3}
ages['Brianna']   # → KeyError!
ages.get('Brianna')# → None (safe)
```

remove by index	del mylist[i]
remove by value	mylist.remove(val)
add to end	mylist.append(val)

⚠ Accessing a missing dict key raises KeyError . Use .get() to avoid it.

SCOPE & VARIABLES

global variable	accessible by all functions
local variable	only inside its function

```
x = 10          # global

def my_func():
  global x      # must declare
  x = 99
```

⚠ Lists are mutable — y = x; y.append(4) also changes x (same object).

OOP — CLASSES

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname  = lname
  def printname(self):
    print(self.firstname, self.lastname)
  def __str__(self):
    return f"{self.firstname} {self.lastname}"

class Student(Person):
  pass          # inherits everything

x = Student("Mike", "Moor")
x.printname()     # → Mike Moor
print(x)          # uses __str__
```

INHERITANCE & POLYMORPHISM

```
class Plant:
  def __init__(self, plant_type):
    self.__plant_type = plant_type  # private
  def get_plant_type(self):
    return self.__plant_type
  def message(self):
    print("I'm a plant.")

class Tree(Plant):
  def __init__(self):
    Plant.__init__(self, 'tree')  # call super
  def message(self):              # override
    print("I'm a tree. I am "
        + self.get_plant_type())

p = Plant('lemon')
t = Tree()
p.message()  # I'm a plant.
t.message()  # I'm a tree. I am tree
print(p)     # uses __str__ → I am a lemon
```

DUNDER / MAGIC METHODS

__init__	constructor — runs on creation
__str__	human-readable string
__repr__	debug/unambiguous string

FILES & MODULES

read file data	called "reading data"
turtle graphics	import turtle
entry guard	if __name__ == '__main__':

Always use import turtle (lowercase). import Turtle is wrong.

TURTLE GRAPHICS

```
import turtle
turtle.hideturtle()
turtle.penup()
turtle.goto(100, 0)     # move to x,y
turtle.fillcolor('blue')
turtle.pendown()
turtle.begin_fill()
for count in range(4):   # draw square
  turtle.forward(50)
  turtle.left(90)
turtle.end_fill()
```

The turtle starts facing right (+x). In the default coordinate system (0,0) is center; positive y is *up*. Calling goto(100,0) moves right; a square drawn counter-clockwise sits in the lower-left relative to the start point.

OOP EXAM Q (PERSON → CLIENT)

```
class Person:
  def __init__(self, name, email, phone):
    self.__name  = name
    self.__email = email
    self.__phone = phone
  def get_name(self):  return self.__name
  def get_email(self): return self.__email
  def get_phone(self): return self.__phone

class Client(Person):
  def __init__(self, name, email,
          phone, cnum, sms):
    super().__init__(name, email, phone)
    self.__cnum = cnum
    self.__sms  = sms   # bool True/False
  def display(self):
    print(f"Name: {self.get_name()}")
    print(f"Address: {self.get_email()}")
    print(f"Phone: {self.get_phone()}")
    print(f"Customer No: {self.__cnum}")
    print(f"On SMS List: {self.__sms}")
```

NUMBER FORMATTING

```
amount = NUM_SHARES * PURCHASE_PRICE
commission = COMMISSION_RATE * amount

print("Amount: €", format(amount, '.2f'))
# → Amount: € 63400.65
```

'.2f'	2 decimal places
f"Hi {name}"	f-string (modern)
named constant	ALL_CAPS by convention

QUICK-FIRE ANSWERS

graphical program steps	flowchart
while loop type	condition-controlled
compound Boolean	compound expression
missing dict key	KeyError
strip from right	rstrip()
first validation input	priming read
find substring	find(substring)
constructor method	__init__
turtle import	import turtle